

Introduction to ClaySharp

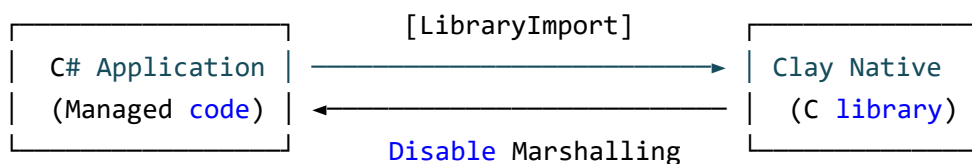
ClaySharp is a modern C# binding for **Clay**, a high-performance 2D layout engine written as a single-header C library. Designed for game engines, desktop shells (such as **BlackHook**), and graphics-intensive applications, ClaySharp allows C# developers to define complex, responsive user interfaces with absolute minimum overhead.

Key Features

- **Zero-Allocation Layout Loop:** Designed from the ground up to prevent heap allocation churn. Dynamic layout trees are computed inside an unmanaged memory block, bypassing C# Garbage Collection completely.
- **Flexbox-like Alignment Rules:** Easily position children horizontally or vertically with customized alignment (left/top, center, right/bottom) and layout spacing gap sizes.
- **Custom Sizing & Aspect Ratios:** Supports flexible sizing constraints including fixed sizing (**Fixed**), matching children bounds (**Fit**), proportioning sibling layouts (**Grow**), or scaling relative to parents (**Percent**), alongside explicit aspect ratio locks.
- **Floating (Absolute) Coordinates:** Design menus, tooltips, dialogs, and panels that float relative to parents, specific elements by ID, or the viewport root with dedicated depth (**ZIndex**) layers.
- **Viewport Culling & Clipping:** Enable boundary clipping and culling elements outside the visible viewport to optimize rendering performance for lists or nested views.
- **Pointer Hover & Scroll Handling:** Seamlessly update mouse/touch inputs and process viewport scrolling offsets, momentum, and click collisions natively.

Architecture & Performance Design

Unlike traditional UI frameworks that utilize deep inheritance trees, reflection, or heavy managed wrappers, ClaySharp focuses on raw ABI compatibility and ultra-fast transitions:



1. **Source-Generated Marshalling (`[LibraryImport]`):** By shifting native marshalling code generation to compile time, C# compiles optimized transition instructions, avoiding reflection overhead entirely.
2. **Runtime Marshalling Disabled:** We disable runtime marshalling via `[assembly: DisableRuntimeMarshalling]`, ensuring C# structures are passed direct bit-for-bit (blitted) without conversions.

3. **No Garbage Collection Transitions:** High-frequency APIs are marked with `[SuppressGCTransition]` to prevent GC overhead on every layout query.

Getting Started with ClaySharp

This guide walks you through the step-by-step process of integrating **ClaySharp** into your C# rendering pipeline.



Integration Steps

Step 1: Memory Arena Allocation

Clay runs within a single block of unmanaged memory (a linear arena). You must query the engine for its minimum allocation size, allocate a backing block, and initialize the **Arena** structure:

```
using System.Runtime.InteropServices;
using ClaySharp.Interop;

// 1. Query minimum size required by the layout engine
uint minMemoryBytes = ClayNative.MinMemorySize();

// 2. Allocate the unmanaged backing memory block
void* arenaMemory = (void*)Marshal.AllocHGlobal((int)minMemoryBytes);

// 3. Create the Arena structure
Arena arena = ClayNative.CreateArenaWithCapacityAndMemory((UIntPtr)minMemoryBytes,
arenaMemory);
```

Step 2: Initialize Context

Initialize Clay with the memory arena, the initial screen dimensions, and a diagnostic error handler:

```
var dimensions = new Dimensions { Width = 1024, Height = 768 };

var errorHandler = new ErrorHandler
{
    ErrorHandlerFunction = Marshal.GetFunctionPointerForDelegate(new
Action<ErrorData>(OnClayError)),
    UserData = null
};

// Initialize the native layout context
ClayNative.Clay_Initialize(arena, dimensions, errorHandler);

// ---
```

```
static void OnClayError(ErrorData error)
{
    string msg = Marshal.PtrToStringAnsi((IntPtr)error.ErrorText.Chars,
error.ErrorText.Length);
    Console.WriteLine($"[Clay Layout Error] {error.ErrorType}: {msg}");
}
```

Step 3: Register Text Measurement Callback

If your user interface displays text elements, Clay must be able to calculate text dimensions. Register a callback that implements your custom text measurement algorithm:

```
// Define the measurement delegate
var measureCallback = new ClayNative.Clay_MeasureTextCallback(MeasureText);

// Register the callback with Clay
ClayNative.SetMeasureTextFunction(Marshal.GetFunctionPointerForDelegate(measureCallback),
null);

// ---

static Dimensions MeasureText(ClayStringSlice text, TextElementConfig* config,
void* userData)
{
    // Access styling using config->FontSize, config->FontId, etc.
    float width = text.Length * (config->FontSize * 0.5f); // Simple fallback
width approximation
    return new Dimensions { Width = width, Height = config->FontSize };
}
```

Step 4: The Frame Layout Loop

During your application update and render frames, feed pointer inputs and execute a layout calculations block. You can declare layout elements using C# resource scopes (`using` blocks):

```
public static unsafe void RunLayoutFrame(float mouseX, float mouseY, bool isLeftMouseDown)
{
    // A. Feed pointer position and button state to Clay
    ClayNative.SetPointerState(new Vector2 { X = mouseX, Y = mouseY }, isLeftMouseDown);

    // B. Begin layout pass calculation
    ClayNative.BeginLayout();
}
```

```

// Declare a root panel declaration
var rootPanel = new ElementDeclaration
{
    Id = ClayNative.GetElementId(CreateClayString("root_panel")),
    BackgroundColor = new Color { R = 20, G = 20, B = 22, A = 255 },
    Layout = new LayoutConfig
    {
        LayoutDirection = LayoutDirection.LeftToRight,
        ChildAlignment = new ChildAlignment { X = LayoutAlignmentX.Center, Y =
LayoutAlignmentY.Center },
        Sizing = new Sizing
        {
            Width = new SizingAxis { Type = SizingType.Grow },
            Height = new SizingAxis { Type = SizingType.Grow }
        }
    }
};

// C. Declare the UI structure
using (new ClayElementScope(in rootPanel))
{
    // Define children here...
}

// D. Finalize calculations and get the output draw commands
RenderCommandArray commands = ClayNative.EndLayout();

// E. Paint the UI using the commands
RenderCommands(commands);
}

```

Step 5: Processing Render Commands

Iterate through the resulting `RenderCommandArray` and draw each item using your graphics backend (e.g. SkiaSharp, OpenGL, Vulkan):

```

static void RenderCommands(RenderCommandArray commands)
{
    for (int i = 0; i < commands.Length; i++)
    {
        RenderCommand* cmd = ClayNative.RenderCommandArray_Get(commands, i);
        if (cmd == null) continue;

        switch (cmd->CommandType)

```

```

{
    case RenderCommandType.Rectangle:
        // Draw filled rectangle using cmd->BoundingBox and
cmd->RenderData.Rectangle
        break;

    case RenderCommandType.Text:
        // Paint text using cmd->BoundingBox and cmd->RenderData.Text
        break;

    case RenderCommandType.ScissorStart:
        // Clip drawing coordinates to cmd->BoundingBox area
        break;

    case RenderCommandType.ScissorEnd:
        // Restore previous clipping boundaries
        break;
}
}
}

```

Namespace ClaySharp.Interop

Classes

[ClayNative](#)

Provides low-level, high-performance C# bindings to the native Clay layout library. Uses source-generated marshalling via [LibraryImportAttribute](#) and compiles with runtime marshalling disabled for zero-overhead, direct native transitions.

Structs

[Arena](#)

Defines a contiguous block of pre-allocated unmanaged memory that Clay uses for all structural and dynamic layout allocations.

[AspectRatioElementConfig](#)

Imposes aspect ratio constraints on a UI element.

[AspectRatioElementConfigArray](#)

Unmanaged array wrapper for [AspectRatioElementConfig](#) elements.

[BorderElementConfig](#)

Configures styling properties for drawing borders around layout nodes.

[BorderElementConfigArray](#)

Unmanaged array wrapper for [BorderElementConfig](#) elements.

[BorderRenderData](#)

Describes calculated border coordinates and stroke styling emitted inside border render commands.

[BorderWidth](#)

Specifies border stroke widths for each edge of a container.

[BoundingBox](#)

Defines a 2D bounding rectangle representing computed UI element placements.

[ChildAlignment](#)

Specifies how child elements are aligned within their parent container along both axes.

[ClayString](#)

A native string wrapper used by Clay to represent text safely across the ABI.

[ClayStringSlice](#)

Represents a slice or segment of a string. Used by Clay's text layout engine during wrapping and text rendering.

[ClipElementConfig](#)

Configures boundary clipping properties for layout nodes.

[ClipElementConfigArray](#)

Unmanaged array wrapper for [ClipElementConfig](#) elements.

[ClipRenderData](#)

Describes clipping parameters emitted inside scissor/clip render commands.

[Color](#)

Represents a color using four floating-point components (Red, Green, Blue, Alpha).

[Context](#)

Opaque wrapper representing Clay's internal layout engine context. This state should only be accessed via pointers returned by [Clay_Initialize\(Arena, Dimensions, ErrorHandler\)](#) or [GetCurrentContext\(\)](#).

[CornerRadius](#)

Specifies corner rounding radii for rectangles, borders, and images.

[CustomElementConfig](#)

Custom element layout configuration block.

[CustomElementConfigArray](#)

Unmanaged array wrapper for [CustomElementConfig](#) elements.

[CustomRenderData](#)

Describes a custom element payload emitted inside custom render commands.

[DebugElementData](#)

Structural debugging attributes associated with layout elements.

[Dimensions](#)

Represents standard 2D floating-point dimensions.

[ElementConfig](#)

Associates a layout configuration style with its category type identifier.

[ElementConfigArraySlice](#)

Represents a slice segment pointing into a native unmanaged array of [ElementConfig](#) entries.

[ElementConfigUnion](#)

Emulates an anonymous configuration C union inside [ElementConfig](#), holding style detail pointers. Only one field should be accessed corresponding to the [Type](#).

[ElementData](#)

Public structure containing calculated layout coordinates.

[ElementDeclaration](#)

Composite declaration structure encapsulating all layout, style, and rendering configs for a single UI element. Pass this to [ConfigureOpenElement\(ElementDeclaration\)](#) to apply styling.

[ElementId](#)

Holds unique identifiers generated by Clay to track element lifecycle, focus, and layout state.

[ElementIdArray](#)

Unmanaged array wrapper for [ElementId](#) elements.

[ErrorData](#)

Structure containing detailed diagnostic information returned when layout errors occur.

[ErrorHandler](#)

Holds callback function pointer registrations for native error handling.

[FloatingAttachPoints](#)

Defines target attachment points for floating elements.

[FloatingElementConfig](#)

Controls positioning, depth, and attachment behavior of floating (absolute positioned) UI elements.

[FloatingElementConfigArray](#)

Unmanaged array wrapper for [FloatingElementConfig](#) elements.

[ImageElementConfig](#)

Holds image element render configurations.

[ImageElementConfigArray](#)

Unmanaged array wrapper for [ImageElementConfig](#) elements.

[ImageRenderData](#)

Describes image content emitted inside image render commands.

[LayoutConfig](#)

Defines general layout configurations for element arrangement, spacing, and sizing.

[LayoutConfigArray](#)

Unmanaged array wrapper for [LayoutConfig](#) elements.

[LayoutElement](#)

Represents a core layout node within Clay's internal structural layout tree.

[LayoutElementChildren](#)

Emulates an anonymous C-style union containing either list of child element indices or text element details. Field offsets overlap so only one field should be accessed corresponding to layout node type.

[LayoutElementChildrenData](#)

Holds list of child elements inside a container.

[LayoutElementHashMapItem](#)

Represents an entry in Clay's internal layout spatial hash map, used for fast lookup, hover, and layout calculation.

[LayoutElementTreeNode](#)

A layout tree node structure tracking hierarchical rendering positioning.

[LayoutElementTreeRoot](#)

Defines the entry root configurations for a layout tree rendering hierarchy.

[MeasureTextCacheItem](#)

Represents an item stored inside Clay's internal text measurement cache.

[MeasuredWord](#)

Holds sizing metadata for a single word computed during text layout wrapping.

[Padding](#)

Defines inner spacing padding values surrounding the children of a container.

[PointerData](#)

Contains pointer (e.g. mouse or touch) position and state information.

[RectangleRenderData](#)

Describes a filled background rectangle emitted inside rectangle render commands.

[RenderCommand](#)

Encapsulates a single drawing instruction emitted by Clay's layout resolver.

[RenderCommandArray](#)

Contiguous unmanaged array containing the list of [RenderCommand](#) elements computed for the current frame.

[RenderData](#)

Emulates an anonymous C union packing specific layout drawing payloads. Only one field should be accessed corresponding to the [CommandType](#).

[ScrollContainerData](#)

Public structure containing scroll coordinates and dimensions for scrollable elements.

[ScrollContainerDataInternal](#)

Internal layout engine scroll tracker holding physics and position metadata for a scroll container.

[SharedElementConfig](#)

Represents style settings that are shared across multiple layout nodes.

[SharedElementConfigArray](#)

Unmanaged array wrapper for [SharedElementConfig](#) elements.

[Sizing](#)

Defines layout sizing rules for both axes of a UI element.

[SizingAxis](#)

Specifies sizing rules and constraints for a single layout axis (either Width or Height).

[SizingAxisValue](#)

Emulates an anonymous C union representing a sizing constraint value. Field offsets overlap so that it can store either percentage values or min/max bounds.

[SizingMinMax](#)

Defines the minimum and maximum constraints for element dimensions.

[StringArray](#)

Unmanaged array wrapper for [ClayString](#) elements.

[TextElementConfig](#)

Provides configuration settings for rendering text elements.

[TextElementConfigArray](#)

Unmanaged array wrapper for [TextElementConfig](#) elements.

[TextElementData](#)

Holds layout and wrapping details computed specifically for text elements.

[TextRenderData](#)

Describes text content and style attributes emitted inside text render commands.

[Vector2](#)

Represents a standard 2D vector.

[Warning](#)

Structure packing diagnostic warning messaging.

[WrappedTextLine](#)

Describes a single computed line of text resulting from word wrapping operations.

[WrappedTextLineArraySlice](#)

Represents a slice segment pointing into an unmanaged array of wrapped text lines.

[boolArray](#)

Unmanaged array wrapper for boolean elements.

[charArray](#)

Unmanaged array wrapper for raw character bytes.

[int32_tArray](#)

Unmanaged array wrapper for 32-bit signed integers.

Enums

[ElementConfigType](#)

Specifies the type of layout configuration applied to a UI element. This configuration determines which rendering, sizing, or layout rules Clay applies to the element.

[ErrorType](#)

Defines the specific errors that can occur during layout calculation or configuration in Clay.

[FloatingAttachPointType](#)

Defines the attachment points on an element and its target when using floating (absolute) positioning.

[FloatingAttachToElement](#)

Identifies which element a floating container attaches to.

[FloatingClipToElement](#)

Controls whether a floating element should be clipped by another element's bounds.

[LayoutAlignmentX](#)

Controls horizontal alignment of children within a layout container.

[LayoutAlignmentY](#)

Controls vertical alignment of children within a layout container.

[LayoutDirection](#)

Defines the layout direction along which children are arranged.

[PointerCaptureMode](#)

Controls pointer capture behavior for floating elements.

[PointerDataInteractionState](#)

Represents the interaction state of a pointer (e.g. mouse or touch) on a UI element.

[RenderCommandType](#)

Specifies the rendering operation command type emitted in the output render command array.

[SizingType](#)

Determines how an element is sized along a layout axis.

[TextAlignment](#)

Specifies the horizontal alignment of text within its bounding container.

[TextElementConfigWrapMode](#)

Defines how text within a text element wraps.

Delegates

[ClayNative.Clay_MeasureTextCallback](#)

Defines the signature of the text measurement callback used by Clay to compute text dimensions.

[ClayNative.OnHoverCallback](#)

Defines the signature of the callback invoked when a pointer hovers over an element configured with an hover handler.